Assignment: Building the `triangdist` R Package

Gherardo Varando

2026

Introduction

In this assignment, you will develop a formal R package named `triangdist` that implements the **Triangular Distribution**. You are required to follow professional software development practices, including version control, automated testing, and comprehensive documentation.

Mathematical Background

The triangular distribution is defined by three parameters: a lower limit a, an upper limit b, and a mode c ($a \leq c \leq b$).

1. The density $f(x)$ increases linearly from 0 at $x = a$ to its maximum value at $x = c$.
2. The density $f(x)$ decreases linearly from its maximum value at $x = c$ to 0 at $x = b$.
3. The total area under the density curve must equal 1.
4. The density is 0 everywhere outside the interval $[a, b]$.

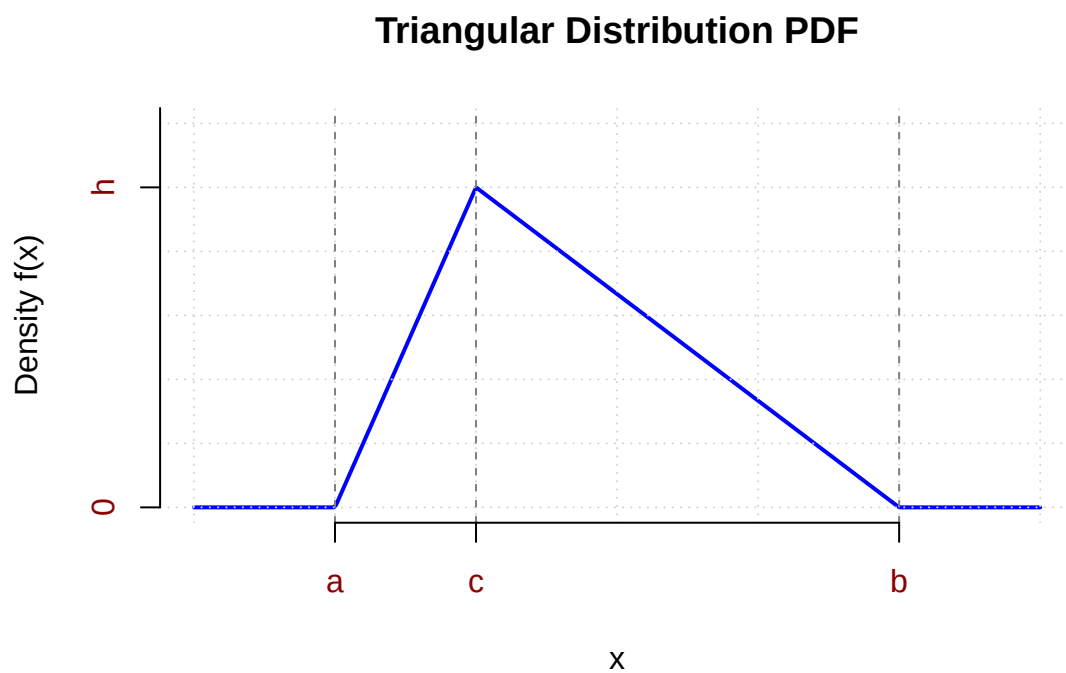

Figure 1: Illustration of the Triangular Density

You can check the wikipedia page for details https://en.wikipedia.org/wiki/Triangular_distribution.

Be careful that you need to derive the formula for the density and you need to compute the hight h as a function of a,b and c.

Implementation Requirements

You must implement the following functions, ensuring they handle **vectorized inputs** for all parameters (using R's recycling rules):

- `dtriang(x, min, max, mode)`: Density function.
- `ptriang(q, min, max, mode)`: Distribution function.
- `qtriang(p, min, max, mode)`: Quantile function.
- `rtriang(n, min, max, mode)`: Random generation.

Hint: Implementing `rtriang`

The most efficient way to implement `rtriang` is via the **Inversion Method** (also known as Inverse Transform Sampling). This method relies on the fact that if U is a uniform random variable on $[0, 1]$, then $X = F^{-1}(U)$ follows the distribution F.

Since you are already implementing `qtriang` (the inverse CDF), your `rtriang` function can be written in a single line of logic:

1. Generate n random values from a standard uniform distribution `runif(n)`.
2. Pass these values as the probability argument `p` into your own `qtriang` function.

This approach ensures that your random number generator is mathematically consistent with your quantile function and stays fully vectorized.

Alternatively you can implement `rtriang` using rejection sampling.

Constraints & Validation

- **Error Handling:** Functions must stop and return clear error messages if `min >= max`, if `mode` is outside $[min, max]$, or if $p \notin [0, 1]$ for `qtriang`.
- **Documentation:** Use `roxygen2` tags (`@param`, `@return`, `@export`, etc.) for all functions. Write documentation in english.
- **Unit Testing:** Use the `testthat` package. You must achieve **>95% code coverage**.

Recommended Workflow

Use the `usethis` package to scaffold your project efficiently:

```r
# Setup commands
usethis::create_package("triangdist")
usethis::use_git()
usethis::use_github()
usethis::use_testthat()
usethis::use_r("triang_dist")
```

Submission

Submit the URL to your **public GitHub repository**. If you do not know how to use git or github this is a perfect occasion to start learning, please see https://docs.github.com/en/get-started/start-your-journey/git-and-github-learning-resources.

Your package must pass `devtools::check()` with no errors or warnings. The `README.md` should include instructions on how to install the package using `remotes::install_github()`.

Basically everything you need to know on how to develop R packages can be found in this amazing book https://r-pkgs.org/.

Quality Control: Coverage and Style

To meet the "95% coverage" and "clean code" requirements, you must use the following tools before your final submission:

Code Coverage with `covr`

The `covr` package tracks which lines of your code are actually executed during your tests. To check your coverage locally:

```r
# Run this to see a detailed report of which lines are tested
covr::report()
```

- **Goal:** Every conditional branch (e.g., both the `if` and `else` parts of your distribution logic) must be exercised by a test case in your `tests/testthat/` files.

Static Analysis and Style with `lintr`

To ensure your code is readable and follows the Tidyverse style guide (e.g., proper spacing, variable naming, and no commented-out code), use `lintr`:

```r
# Run this to find stylistic "errors" or potential bugs
lintr::lint_package()
```

- **Requirement:** Address all "Error" and "Warning" level lints. A professional package should be as clean as it is functional.

Comprehensive Check

Finally, run a full build check. This simulates how CRAN (the official R package repository) would inspect your work:

```r
devtools::check()
```

- **Submission Criteria:** Your package must return **0 errors, 0 warnings, and 0 notes**.